Blockchain Transaction Verification

22AIE205

Group-12

Lakshmi Tejaswi - CB.SC.U4AIE24116

K. Kiranmai - CB.SC.U4AIE24119

K. Sudeshna - CB.SC.U4AIE24120

INTRODUCTION

This project presents a Blockchain Transaction Management and Fraud Detection System developed in Python. It efficiently stores, searches, and deletes blockchain transactions using a Hash Table with Linked List chaining for collision handling. Each transaction is identified by a unique Transaction Hash (TxHash) and stored persistently in CSV files. By combining data structures like Linked Lists and Hash Tables with practical blockchain concepts, the project demonstrates how secure and efficient transaction management can be achieved through structured data organization

OBJECTIVES:

- To efficiently store, search, and delete blockchain transactions using a hash-based system.
- To implement a Hash Table with Linked List chaining for collision handling.
- To develop a fraud detection module that identifies suspicious activities such as self or duplicate transactions.
- To provide options to view hash table indexes, count elements per index, and display the entire hash table.

DATA STRUCTURES USED :

- Linked List: stores transactions in each hash bucket.
- Hash Table: maps TxHash → bucket index using modular hashing.

Why these structures were chosen?

- Fast lookup: Hashing allows direct access to the correct bucket.
- Efficient insertion/deletion: Linked List chaining handles collisions without shifting data.

METHODOLOGY:

1. Hash Function:

- Each TxHash (Transaction Hash) is first manually converted from hexadecimal to integer using the manual_hex_to_int() function.

- The integer value is then used to compute an index:

index = int % table_size

- This ensures that each transaction is assigned to a index in the hash table.

2. HashTable Operation :

- Transaction Insertion

- Delete a Transaction
    1. Remove transaction from Hash Table.
    2. Move deleted transaction to deleted_transactions.csv

- Search a Transaction
    1. Search by TxHash in Hash Table.

- Finding Index of specific Txhash

- Count elements in specific index

- Display HashTable

3. Fraud Detection :

Fraud detection function checks transactions across both active and deleted files.

- ***Self-Transaction:***
  Flags transactions where the sender and receiver are the same (From == To).

- ***High-Value Transaction:***
  Flags transactions with a value greater than 5 units (customizable threshold).

- ***High-frequency transactions from same sender***
  A sender sending too many transactions in total (more than or equal to 5 here) may indicate money laundering, spamming, or abnormal activity.

- ***Duplicate From–To–Value Transaction:***
  Flags transactions where the sender, receiver, and value are identical to another transaction.

ADVANTAGES:

- Efficient lookup and storage using hash table + linked lists.
- Handles collisions.
- Immediate fraud detection on insertion and search.
- Scalable for large datasets.

RESULTS:

```
Options:
1. Add a new transaction
2. Delete a transaction
3. Search a transaction
4. Exit
5. Find hash table index of a transaction
6. Count elements at an index
7. Display entire hash table
8. Fraud check for all transactions (active + deleted)
Enter your choice (1-8):
```

we need to enter a choice (1–8) depending on what we need

CONCLUSION:

This system efficiently manages blockchain transactions using hash tables and linked lists, enabling fast insertion, search, and deletion. Fraud detection identifies self-transactions, high-value transfers, and duplicate transactions across active and deleted datasets. By combining optimized data structures with systematic checks, it ensures secure, scalable, and reliable transaction management.

Thank You